

School of Engineering, Applied Sciences and Technology

Paradigm Analysis of the Quickselect Algorithm

Divide and Conquer with a Greedy (Always-Median Pivot Strategy) Approach

Student: Leanne Jessica Rodrigo
Student ID: 20210001983
Course: BCS 309 — Algorithms I
Instructor: Dr. Arash Kermani
Submission Date: April 25, 2026

Contents

1	Introduction	1
2	Problem Statement	1
3	Algorithm 1: Divide and Conquer	2
3.1	Sorting and searching component	2
3.2	Divide and conquer pseudocode	3
3.3	Why divide and conquer fits the problem	3
4	Algorithm 2: Greedy	3
4.1	Greedy pseudocode	3
4.2	Why greedy works and its limitation	4
5	Worked Example	4
5.1	Quickselect solution	4
5.2	Greedy solution	6
6	Comparison of Divide and Conquer and Greedy	6
7	Complexity and Recursion Analysis	7
7.1	Time complexity	7
7.2	Space complexity	8
7.3	Recursion behavior	8
8	Conclusion	9

1. Introduction

The selection problem asks for the element of a specified rank in an unsorted collection without requiring the entire collection to be arranged in order. This problem is important because many practical tasks need only one ranked value, such as the minimum qualifying score, the median reading from a sensor stream, or the fifth-fastest completion time in a race. In such situations, full sorting performs extra work by ordering values whose exact relative positions are irrelevant to the final answer. This report studies **Quickselect**, a **Divide and Conquer** algorithm that solves the selection problem by partitioning the array around a pivot and then recursing only into the side that can still contain the required rank. The report also introduces a **Greedy (Always-Median Pivot Strategy)** alternative in order to contrast a locally optimal pivot-choice strategy with a structurally reducing recursive strategy. The tool is available at: <https://leannejrodrigo.github.io/quickselect-visualizer/>, and this report is derived from the behavior of the above Quickselect visualizer; all pseudocode and execution traces directly reflect its implementation. Accordingly, the discussion in this document follows the exact behavior of a Quickselect implementation that always chooses the last element as the pivot, applies Lomuto partitioning, uses the condition $A[j] \leq \text{pivot}$, works in place on the array, converts the input rank from k to $k - 1$, and recurses on only one side after partitioning.

2. Problem Statement

The problem A is the Selection Problem. The office does not need the full list sorted from smallest to largest. Instead, it needs to identify the score that is the k -th smallest so that a rank threshold can be set quickly. This is a realistic selection problem because the office is interested in one specific rank rather than the complete ordering of every applicant. The input to the problem is an **array** A of numbers together with a positive integer k , where k is given in **1-based** form. The required output is the value that would appear in position k if the array were sorted in nondecreasing order. In the Quickselect visualizer used as the reference for this report, the user-supplied rank is converted to zero-based indexing before selection begins, so the implementation actually searches for index $k - 1$. The algorithm used is **Quickselect**, and its governing paradigm is **Divide and Conquer** because each partition step places one element permanently and then reduces attention to only the subproblem that can still contain the desired rank.

To make the task concrete, let

$$A = [3, 6, 8, 5, 2, 7, 1, 4] \quad \text{and} \quad k = 4.$$

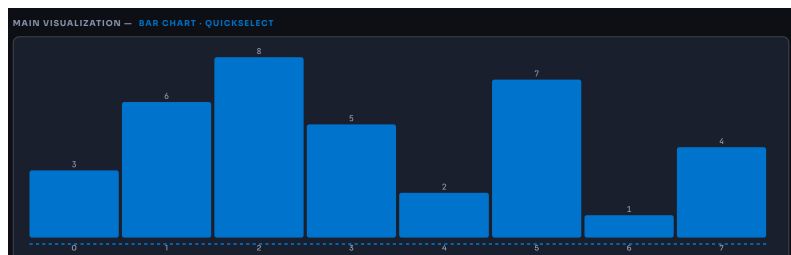


Figure 1: Initial state of the array before Quickselect begins

The required zero-based target is therefore $k - 1 = 3$, meaning that the algorithm must find the element that belongs at index 3 after ordering. The execution begins on the full array with `lo = 0` and `hi = 7`. Because the implementation always chooses the last element as the pivot, the first pivot is 4. Lomuto partitioning scans from left to right and moves values less than or equal to the pivot toward the front. The values 3, 2, and 1 are less than or equal to 4, so after the loop the pivot is swapped into index 3.

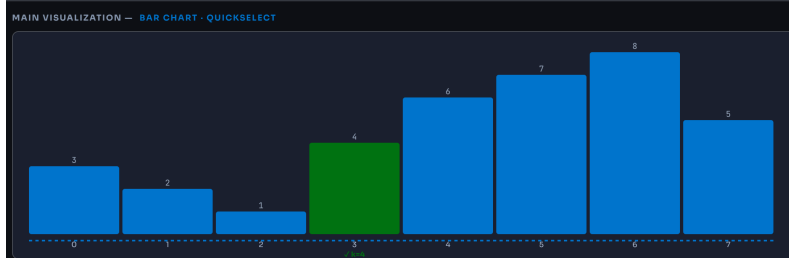


Figure 2: First partition step where pivot 4 is placed at index 3

The array becomes $[3, 2, 1, 4, 6, 7, 8, 5]$, and the pivot index is 3. Since the target index is also 3, the algorithm stops immediately and returns 4. Therefore, the fourth smallest element in the original unsorted array is 4. This trace illustrates the essence of the problem: the method finds the required rank by repeatedly shrinking the relevant part of the array rather than sorting every value.

3. Algorithm 1: Divide and Conquer

3.1 Sorting and searching component

Quickselect is best understood as a **partition-based searching** method rather than a full sorting method. Its purpose is not to produce a completely ordered array; instead, it searches for the position where the required rank must lie. The key searching action is the partition operation, which places the pivot in its final sorted position and separates the active array into values that are less than or equal to the pivot and values that are greater than the pivot. The partition procedure used by the visualizer is shown below. The uploaded HTML confirms that the tool uses the standard Lomuto boundary variable with $i \leftarrow lo - 1$ before the scan begins. The pseudocode below summarizes this partition step before the visual trace is shown.

```

1. PARTITION(A, lo, hi)
2.   pivot  $\leftarrow A[hi]$ 
3.    $i \leftarrow lo - 1$ 
4.   for  $j \leftarrow lo$  to  $hi - 1$ 
5.     if  $A[j] \leq \text{pivot}$ 
6.        $i \leftarrow i + 1$ 
7.       swap  $A[i], A[j]$ 
8.   swap  $A[i+1], A[hi]$ 
9.   return  $i + 1$ 

```

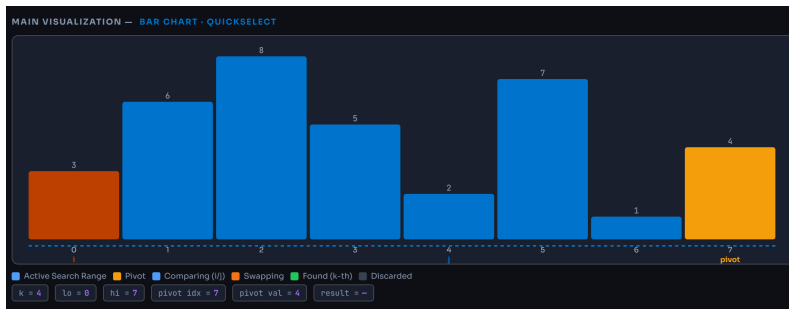


Figure 3: Lomuto partition process showing element swaps during execution

An alternative approach would be to sort the entire array first, for example by using Merge Sort, and then read the element at position $k - 1$. That approach is correct, but it is less well aligned with the actual objective of the problem. Quickselect is better because it focuses directly on *rank finding*. After each partition, one element is fixed in its final position and one whole side can be ignored. By contrast, a full sorting method continues arranging elements even when their detailed order does not affect the answer. Quickselect therefore matches the problem goal more precisely: it searches for one ranked value instead of organizing the whole dataset. The data properties in this problem strongly support the use of partitioning. First, the input is **unsorted**, so the algorithm must be able to work without any prior ordering. Second, the array may contain **duplicates**, and the condition $A[j] \leq \text{pivot}$

correctly groups repeated values with the pivot side during Lomuto partitioning. Third, the values may arrive in **arbitrary order**, meaning that there is no structure to exploit through simple scanning rules alone. Partitioning fits these properties because it imposes exactly the local structure that selection needs: it identifies a pivot's final rank and separates smaller-or-equal values from larger values in place. Sorting can still be the better choice when many order statistics or the full final order will be queried repeatedly after one preprocessing stage, because the one-time effort of building a completely ordered array can then support many later lookups.

3.2 Divide and conquer pseudocode

The Quickselect procedure below is reproduced to match the reference visualizer exactly. Reading it from top to bottom shows the full cycle clearly: partition once, compare the pivot index with the target, and continue only with the relevant side.

```

1. QUICKSELECT(A, lo, hi, k)
2.   if lo = hi
3.     return A[lo]
4.   p ← PARTITION(A, lo, hi)
5.   if p = k
6.     return A[p]
7.   else if k < p
8.     return QUICKSELECT(A, lo, p-1, k)
9.   else
10.    return QUICKSELECT(A, p+1, hi, k)

```

This version matches the tool behavior in every required detail. The pivot is always $A[hi]$, partitioning is Lomuto partitioning, the comparison used inside the loop is $A[j] \leq \text{pivot}$, the array is modified in place, the base case is $lo = hi$, the user rank is converted from k to $k - 1$ before selection starts, and recursion continues on only one side depending on whether the target index is left or right of the pivot position.

3.3 Why divide and conquer fits the problem

Quickselect is a strong fit for the **Divide and Conquer** paradigm because partitioning creates a meaningful structural split. After a pivot is placed, every value on one side is known to be less than or equal to it and every value on the other side is known to be greater. This means that the pivot's rank is final and no later step can change that position. Once this property is established, the algorithm does not need to solve both subproblems. It needs only the side that still contains the target index. The example from Section 1 demonstrates this property directly. In that trace, pivot 4 finishes at index 3, which is exactly the target index, so the algorithm stops immediately without exploring either side further. In other words, Divide and Conquer fits because partitioning produces subproblems, the pivot certifies one exact rank immediately, and the desired answer is isolated by recurring only into the structurally relevant subarray when a further call is needed. The problem shrinks not merely by removing one chosen element, but by preserving a rank boundary that justifies discarding an entire region of the array.

4. Algorithm 2: Greedy

The alternate algorithm B uses the Greedy paradigm. A greedy algorithm makes a sequence of locally optimal choices, hoping that repeated local correctness will produce the required global result. In this report, the local choice is to use a median or approximate median of the current active subarray as the pivot at each step. This paradigm is explicitly different from Divide and Conquer. Divide and Conquer reduces a problem by splitting around a structural boundary and continuing through recursive subproblems, whereas this greedy strategy proceeds iteratively: at each stage it chooses a pivot that is intended to be centrally informative, partitions the current active region, and immediately keeps only the side that still contains the target rank. For a beginner, the key contrast is that Quickselect describes the reduction recursively, while this greedy alternative describes it as an iterative sequence of pivot choices.

4.1 Greedy pseudocode

A greedy method for the selection problem can repeatedly choose a median-based pivot for the current active subarray, partition around that pivot, and then continue only with the side that still contains the required rank. The pseudocode below makes that iterative narrowing process explicit.

```

1. GREEDY-MEDIAN-SELECT(A, k)
2.   lo  $\leftarrow$  0; hi  $\leftarrow$  n - 1
3.   target  $\leftarrow$  k - 1
4.   while lo  $\leq$  hi
5.     m  $\leftarrow$  CHOOSE-MEDIAN-PIVOT(A, lo, hi)
6.     swap A[m], A[hi]
7.     p  $\leftarrow$  PARTITION(A, lo, hi)
8.     if p = target
9.       return A[p]
10.    else if target < p
11.      hi  $\leftarrow$  p - 1
12.    else
13.      lo  $\leftarrow$  p + 1

```

4.2 Why greedy works and its limitation

This greedy method is correct in the sense that each step makes a locally informative pivot choice and then uses the partition result to preserve only the subarray that can still contain the target rank. Once the median-based pivot is placed into its partition position, every value on one side is known to be smaller and every value on the other side is known to be larger. The algorithm then greedily keeps the side whose rank interval still contains k . Repeating this local decision eventually isolates the required index. The local rule is therefore not “choose the next output element,” but rather “choose a pivot that is expected to divide the current search region effectively and immediately continue with the relevant side.” However, the limitation of the greedy approach is that its performance depends on the quality and cost of the median estimate used at each step. If the chosen pivot is close to the true median, the active region may shrink quickly. If the estimate is poor, the reduction can be much less effective. A concrete example appears with $A = [3, 6, 8, 5, 2, 7, 1, 4]$ and $k = 4$. If the algorithm first chooses a central pivot such as 5, partitioning can immediately separate the array into values below and above that pivot, after which only the side containing the fourth-smallest element is kept. The greedy method therefore does reduce the search space iteratively, but unlike Quickselect it is characterized here by a locally chosen median-style pivot heuristic rather than by a recursive divide-and-conquer structure. The greedy approach is therefore not always guaranteed to be optimal in performance. For example, with $A = [1, 2, 3, 4, 5, 6, 7, 8, 9]$ and $k = 1$, a median-based pivot such as 5 is locally reasonable but does not target the extreme rank directly. The partition is correct, yet the algorithm may discard only part of the array and continue through several further reductions before reaching the minimum. This shows that a locally attractive pivot choice can still lead to inefficient reduction when the target rank lies near an edge.

5. Worked Example

5.1 Quickselect solution

The same example from Section 1 is used here: $A = [3, 6, 8, 5, 2, 7, 1, 4]$ with $k = 4$, so the target index is 3. The data structure is an **array**. This is the correct choice because Quickselect relies on indexed access, in-place swaps, and contiguous active subarrays identified by **lo** and **hi**. An array supports each of these operations directly. The algorithm does not need extra structures to hold smaller and larger values, because partitioning rearranges the existing entries within the same storage. The following table records the complete step-by-step execution. It shows the pivot selected at each stage, the index where that pivot finishes after partitioning, the directional decision, and the changing active and discarded regions. The table makes the divide-and-conquer pattern visible: after every partition, the problem shrinks to one smaller subarray while the other side is permanently discarded.

STEP HISTORY (CLICK TO JUMP)			
#	Action	Pivot	Cmps
1	call	—	7
2	pivot-chosen	4	7
3	compare	4	1
4	swap	4	1
5	compare	4	2
6	compare	4	3

STEP HISTORY (CLICK TO JUMP)			
#	Action	Pivot	Cmps
6	compare	4	3
7	compare	4	4
8	compare	4	5
9	swap	4	5
10	compare	4	6
11	compare	4	7

11	compare	4	7
12	swap	4	7
13	partition-done	4	7
14	found	4	7

Figure 4: Step history progression

Detailed partition steps for Quickselect on $A = [3, 6, 8, 5, 2, 7, 1, 4]$ and $k = 4$

Step	Pivot	Pivot Index	Action	Array State
1	4	—	Initialize partition with pivot $= A[7]$, set $i = -1$	$[3, 6, 8, 5, 2, 7, 1, 4]$
2	4	—	Compare $A[0] = 3$ with pivot; $3 \leq 4$ is true	$[3, 6, 8, 5, 2, 7, 1, 4]$
3	4	—	Move i to 0 and swap $A[0]$ with $A[0]$	$[3, 6, 8, 5, 2, 7, 1, 4]$
4	4	—	Compare $A[1] = 6$ with pivot; $6 \leq 4$ is false	$[3, 6, 8, 5, 2, 7, 1, 4]$
5	4	—	Compare $A[2] = 8$ with pivot; $8 \leq 4$ is false	$[3, 6, 8, 5, 2, 7, 1, 4]$
6	4	—	Compare $A[3] = 5$ with pivot; $5 \leq 4$ is false	$[3, 6, 8, 5, 2, 7, 1, 4]$
7	4	—	Compare $A[4] = 2$ with pivot; $2 \leq 4$ is true	$[3, 6, 8, 5, 2, 7, 1, 4]$
8	4	—	Move i to 1 and swap $A[1]$ with $A[4]$	$[3, 2, 8, 5, 6, 7, 1, 4]$
9	4	—	Compare $A[5] = 7$ with pivot; $7 \leq 4$ is false	$[3, 2, 8, 5, 6, 7, 1, 4]$
10	4	—	Compare $A[6] = 1$ with pivot; $1 \leq 4$ is true	$[3, 2, 8, 5, 6, 7, 1, 4]$
11	4	—	Move i to 2 and swap $A[2]$ with $A[6]$	$[3, 2, 1, 5, 6, 7, 8, 4]$
12	4	3	Place pivot by swapping $A[3]$ with $A[7]$	$[3, 2, 1, 4, 6, 7, 8, 5]$

Step-by-step Quickselect execution for $A = [3, 6, 8, 5, 2, 7, 1, 4]$ and $k = 4$

Step	Pivot	Pivot Index	Decision (left/right)	Partition Result and Discarded Subarray	Remaining Active Subarray
1	4	–	Start with full call on indices 0 to 7	Active array before partition: $[3, 6, 8, 5, 2, 7, 1, 4]$. No discarded subarray yet.	Indices 0 to 7: $[3, 6, 8, 5, 2, 7, 1, 4]$
2	4	–	Partition scan begins on the full active subarray	During partition, values 3, 2, and 1 are moved left of the pivot boundary.	Indices 0 to 7: intermediate state leads to $[3, 2, 1, 5, 6, 7, 8, 4]$ before final pivot placement
3	4	3	Stop, because $3 = 3$	Result: $[3, 2, 1, 4, 6, 7, 8, 5]$. Pivot reaches the target index, so no left or right subarray is pursued or discarded after the decision.	No further subarray; answer is 4.

The table also highlights pivot movement clearly. In Step 1, pivot 4 moves from the last position into index 3. This movement produces the partition result, identifies the target index immediately, and leaves no remaining active subarray for a further recursive call.

The first two levels of the recursion tree for this example are shown below:

Level 0: $A[0..7]$, size = 8

↓ after partition with pivot 4 at index 3

Level 1: stop; target index 3 equals pivot index 3, so no further recursive call is needed

5.2 Greedy solution

The same example can also be traced with the greedy method on the original unsorted array $A = [3, 6, 8, 5, 2, 7, 1, 4]$ and $k = 4$. The greedy rule is to choose a median-based pivot from the current active region, partition around it, and then keep only the side that still contains the target index. For this array, a natural first local choice is the median-style pivot 5. After partitioning around 5, the values smaller than 5 lie to its left and the larger values lie to its right, so 5 settles beyond the target rank. The algorithm therefore keeps only the left side containing $[3, 2, 1, 4]$. On this reduced region, the next median-style pivot can be 3. Partitioning around 3 places it below the target index, so the algorithm keeps the right side of that active region. The remaining candidates are then $[4]$, so the process stops and returns 4. This trace shows both the strength and the weakness of the greedy approach. Its local rule is easy to understand and it does reduce the search space by keeping only one partition side after each pivot choice. However, the method is still described as greedy rather than divide and conquer because the emphasis is on making the next locally attractive pivot choice and continuing iteratively, rather than on framing the process as a recursive decomposition of subproblems in the Quickselect style. This trace shows both the strength and the weakness of the greedy approach. Its local rule is easy to understand and it does reduce the search space by keeping only one partition side after each pivot choice. However, the method is still described as greedy rather than divide and conquer because the emphasis is on making the next locally attractive pivot choice and continuing iteratively, rather than on framing the process as a recursive decomposition of subproblems in the Quickselect style. In that sense, Quickselect and the greedy method may both discard irrelevant values, but Quickselect explains the reduction recursively while the greedy method explains it as iterative narrowing.

6. Comparison of Divide and Conquer and Greedy

For **small input**, the winner is **Greedy**. On a very small array, a median-style local pivot choice is easy to make and the active region becomes small after only a few iterations. In paradigm terms, the greedy method benefits from the fact that local pivot decisions are easy to validate when the dataset is tiny, so the iterative reduction is straightforward to carry out. For **general input**, the winner is **Quickselect**. The divide-and-conquer strategy is superior because each partition creates structural information about rank and then throws away the irrelevant side through an explicitly recursive formulation. This matches the selection problem at a conceptual level: the algorithm narrows the search to the only subproblem that can still contain the answer. The greedy median-pivot heuristic is also capable of iterative reduction, but its identity comes from repeated local pivot decisions rather than from the same recursive divide-and-

conquer structure. For the case $k = 1$, the winner is still **Quickselect**. Even though the target is the smallest element, Quickselect’s partition logic remains a more direct and dependable paradigm match for rank selection in this report because its behavior is fully specified by the reference implementation. The greedy always-median strategy can still reduce the active region iteratively, but choosing a central pivot is not especially tailored to isolating the minimum. A surprising case is $k = 1$, because one might expect a greedy paradigm to be most natural for an extreme rank, yet the median-pivot greedy rule is not specially aligned with finding the minimum, whereas Quickselect still matches the selection structure more directly. The following table brings the comparison together by placing the two paradigms side by side.

Comparison of Quickselect and Greedy for the selection problem

Paradigm	Strategy	Time complexity behavior	Problem-solving style	Strength for Selection Problem
Quickselect (Divide and Conquer)	Partition around a pivot and recurse only into the relevant side	Usually shrinks quickly, but can become highly unbalanced in the worst case	Structural reduction through partitioning and recursive narrowing	Strong because it targets one rank directly and discards irrelevant regions
Greedy	Repeatedly choose a median-based pivot heuristic and continue with the relevant side iteratively	Depends on the cost and quality of median estimation, with improvement when pivots are centrally placed	Local pivot choice repeated without the same recursive subproblem formulation	Correct as a heuristic selection strategy, but less directly tied to the reference Quickselect structure

7. Complexity and Recursion Analysis

7.1 Time complexity

The Quickselect visualizer implements a version of selection in which each call performs one Lomuto partition on the current active subarray and then recurses on only one side. Because partitioning scans the active range once, the work of a single call is proportional to the size of that range. When the pivot tends to land reasonably close to the target rank, the active subarray shrinks quickly, so the total amount of work behaves like a decreasing series over progressively smaller ranges. In practical terms, this is why Quickselect is usually fast for rank finding. The same tool behavior also reveals the unfavorable case. Since the pivot is always chosen as the last element, already sorted or reverse-sorted data can produce very unbalanced partitions. In such a case, the algorithm may reduce the active range by only one position at a time, forcing many nearly full scans before the answer is found. The greedy alternative has a different pattern: at each stage it must determine a median or approximate median for the current active subarray and then partition around that choice. When those pivots are centrally placed, the active region can shrink effectively; when they are not, the improvement is weaker. Its efficiency therefore depends both on the cost of median estimation and on how balanced the resulting partitions are. A concrete worst-case example is $A = [1, 2, 3, 4, 5, 6, 7, 8]$ with $k = 1$. Because the array is already sorted and the implementation always chooses the last element as the pivot, the first pivot is 8, then 7, then 6, and so on, while the target remains the leftmost index. Each partition therefore removes only one element from the right side and leaves another nearly full scan for the next call, illustrating the worst-case behavior directly.

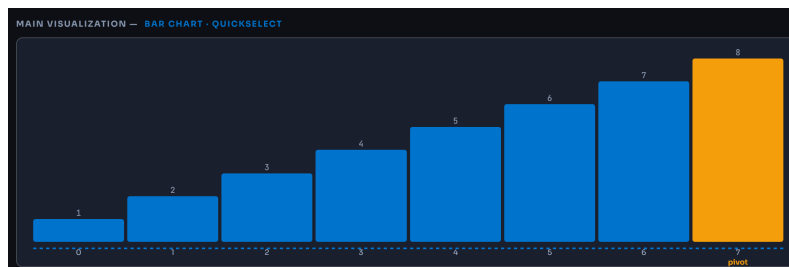


Figure 5: Worst-case Quickselect partitioning with last-element pivot causing highly unbalanced splits

7.2 Space complexity

The main data structure in the visualizer is the array itself. Quickselect works in place, so it does not allocate separate left and right arrays during partitioning. This keeps the extra storage cost low. Beyond the array, the algorithm uses only a small amount of additional information in each call, such as the indices lo , hi , k , the pivot, and the partition boundary. The extra memory cost therefore comes mainly from recursion. Since Quickselect recurses on only one side, the call stack stores a single chain of active subproblems rather than a branching tree of simultaneous recursive calls. On balanced inputs this chain remains short, while on highly unbalanced inputs it can grow much longer. The greedy always-median strategy, by contrast, can be implemented iteratively and therefore avoids a recursive call stack, although it may need additional work space depending on how the median or approximate median is computed at each step.

7.3 Recursion behavior

The recursion shown by the tool is especially important because it reflects the divide-and-conquer logic directly. Each call begins with a range $A[lo..hi]$, partitions that range around the last element, and then keeps only the side that can still contain the target index. If the pivot lands exactly at the target index, recursion stops immediately. If the active subarray is reduced to a single element so that $lo = hi$, the base case returns that value directly. This means that the recursion depth depends entirely on how effectively the pivot narrows the problem. In the worked example, each recursion level represents one active subarray together with the single partition decision made on that subarray. The active range shrinks from seven elements to five, then to four, then to three, then to two, and finally stops when the pivot reaches the target index. The call stack is therefore easy to follow and each recursive step has a clear justification. In harder inputs with consistently poor pivots, the same one-sided recursion can become a long chain because each partition removes very little of the remaining search region. That is why the worst-case recursion becomes deep: the algorithm keeps solving almost the same problem size again and again. The screenshot below illustrates the call stack for the worked example before the worst-case chain is discussed.

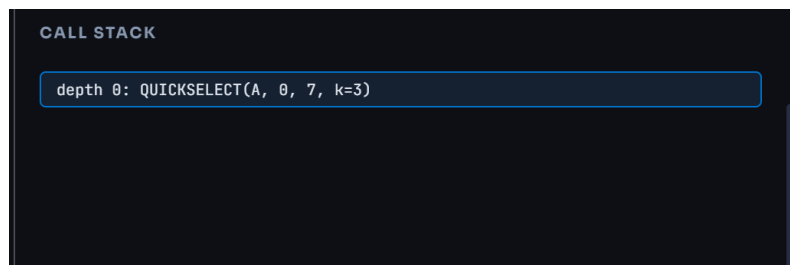


Figure 6: Worked example call stack showing

For the worst-case example $A = [1, 2, 3, 4, 5, 6, 7, 8]$ with $k = 1$, the recursion forms a long one-sided chain: the algorithm first recurses on indices 0 through 6, then 0 through 5, then 0 through 4, and continues in the same pattern until only the first element remains. This makes the recursion depth much larger than in the balanced case and clearly shows how poor pivot placement affects execution. The final screenshot below makes this deep one-sided chain easy to recognize at a glance.

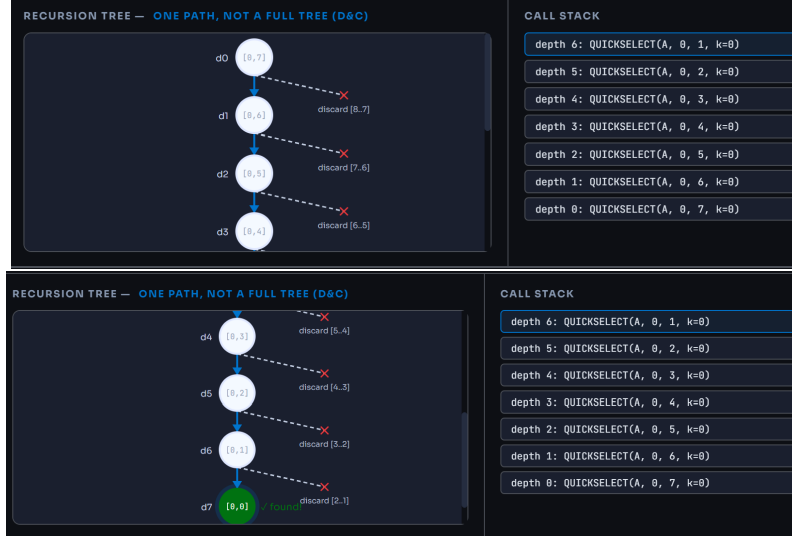


Figure 7: Worst-case recursion tree showing deepening call stack in Quickselect

8. Conclusion

The selection problem matters because many real tasks need one ranked value from an unsorted dataset without requiring the full order of all values. Examples include threshold setting, median extraction, and ranked decision support. In these settings, the essential objective is not to sort everything, but to identify the element with the required rank efficiently and correctly. This report has shown that **Quickselect** is the correct primary paradigm-based solution because it applies **Divide and Conquer** in a way that directly matches the structure of the problem. Each partition places the pivot into its final rank, reduces the problem size efficiently, and focuses attention only on the side that can still contain the required index. The algorithm therefore concentrates on the required rank rather than expending effort on irrelevant order information. The **Greedy** alternative is also correct, but it is weaker as a general strategy for this problem. Its local correctness comes from repeatedly making a median-based pivot decision for the current active region, yet its effectiveness depends on the quality of that pivot heuristic and it is less directly aligned with the reference implementation than Quickselect. The key insight is therefore the contrast between *local correctness* and *structural reduction*: greedy selection succeeds by making the next locally attractive pivot decision, whereas divide and conquer succeeds by transforming the search space into a smaller and more informative subproblem. Finally, the Quickselect visualizer strengthens understanding because it makes the algorithm's behavior visible. By showing pivot movement, discarded subarrays, and the remaining active subarray after each partition, the tool clarifies why the divide-and-conquer paradigm is not merely a description of the code but the central reason the method works so effectively for selection.